DSA -

🧭 Topic: Time & Space Complexity (Big O Notation)

What is Time Complexity?

It measures **how the runtime of an algorithm grows** as the size of the input (n) increases.
It's not the exact number of seconds — it's the *rate of growth*.

Think of it as:

"If I double the input size, how much slower does my algorithm get?"

📈 2️⃣ Big O Notation — The Asymptotic Upper Bound

Big O gives the *worst-case* growth rate.
It ignores constants and less significant terms.

Example:

```
for (int i = 0; i < n; i++) {
   System.out.println(i);
}
```

- Runs n times → **O(n)**

```
for (int i = 0; i < n; i++) {
   for (int j = 0; j < n; j++) {
      System.out.println(i + " " + j);
   }
}
```

- Runs n * n times → **$O(n^2)$**

⚡ Common Time Complexities

Big O	Name	Example
O(1)	Constant	Accessing element by index (arr[i])
O(log n)	Logarithmic	Binary search
O(n)	Linear	Single loop over array
O(n log n)	Log-linear	Merge sort, Quick sort (average)
O(n^2)	Quadratic	Nested loops
O(2^n)	Exponential	Recursive subset generation
O(n!)	Factorial	Generating all permutations

🧱 3️⃣ How to Calculate Big O

Rule 1: Drop constants
O(2n) → **O(n)**

Rule 2: Keep the highest-order term
O(n^2 + n + 1) → **O(n^2)**

Rule 3: Nested loops multiply

```
for(i=0;i<n;i++){
  for(j=0;j<m;j++){
     // O(1)
  }
}
```

→ **O(n × m)**

Rule 4: Sequential code adds

```
for(i=0;i<n;i++){}      // O(n)
for(j=0;j<m;j++){}      // O(m)
```

→ **O(n + m)**

🧠 4️⃣ Best, Average, Worst Case

Case	Description	Example
Best	Minimum time taken	Finding first element that matches
Average	Expected time	Hash lookup with moderate collisions
Worst	Maximum time	Searching for absent element

Interviewers almost always ask for **Worst-case complexity**.

💾 5️⃣ Space Complexity

Measures how much **extra memory** (RAM) the algorithm needs.

Examples:

```
int sum = 0;       // O(1)

int[] arr = new int[n]; // O(n)
```

Key factors:

- Variables
- Recursion stack
- Data structures created at runtime

💡 Examples

Example 1 — Sum of array

```
int sum = 0;

for (int i = 0; i < n; i++) {

  sum += arr[i];

}
```

- Time: **O(n)** (one loop)

- Space: **O(1)** (just one variable)

Example 2 — Recursion (Fibonacci)

```
int fib(int n){
  if(n<=1) return n;
  return fib(n-1) + fib(n-2);
}
```

- Time: **$O(2^n)$**
- Space: **O(n)** (recursion depth)

Example 3 — Merge Sort

- Time: **O(n log n)**
- Space: **O(n)** (for temp arrays)

🧩 7️⃣ **Visual Intuition**

n	O(1)	O(log n)	O(n)	O(n log n)	$O(n^2)$
1	1	0	1	0	1
10	1	3	10	33	100
100	1	6	100	664	10 000
1000	1	10	1000	9966	1 000 000

🧠 Takeaway:
For large n, O(n log n) is *much* better than $O(n^2)$.

How to Think in Interviews

- Identify **input size** (n)
- Count how many times your main operation runs
- Ignore constants and less important parts

- Clarify if the interviewer wants *time* or *space* complexity
- Discuss **trade-offs** (e.g., faster time vs. more space)

Arrays — Prefix Sum & Sliding Window

(Part 2:

⚙️ 1️⃣ PREFIX SUM — Concept

💡 What It Is

Prefix Sum is about **precomputing cumulative sums** so you can get any subarray sum in **O(1)** instead of O(n).

📘 Formula

For an array arr[] of size n:
prefix[i] = prefix[i-1] + arr[i]

Then,

Sum of subarray from l to r = prefix[r] - prefix[l-1]

🧮 Example

int[] arr = {2, 3, 4, 5};

prefix = [2, 5, 9, 14]

// Sum(1,3) => prefix[3] - prefix[0] = 14 - 2 = 12

⚡ When to Use PREFIX SUM

Use it when you hear questions like:

🔍 Problem Type	🔑 Clue Phrases	🚀 Approach
Subarray sum queries	“Find sum of elements from L to R multiple times”	Precompute prefix sum, query in O(1)
Subarray with given sum	“Find number of subarrays with sum = K”	Use prefix + HashMap
Difference arrays	“Add X to range [L, R] repeatedly”	Use prefix diff array trick
2D Grid sum queries	“Sum of rectangle in matrix”	2D prefix sum
Balancing or equal partition	“Split array into equal halves / equal sum parts”	Prefix sum comparison

---

🧠 Example: Subarray Sum Equals K

```
int subarraySum(int[] nums, int k) {
  Map<Integer, Integer> map = new HashMap<>();
  map.put(0, 1);
  int sum = 0, count = 0;
  for (int num : nums) {
    sum += num;
    if (map.containsKey(sum - k))
      count += map.get(sum - k);
    map.put(sum, map.getOrDefault(sum, 0) + 1);
  }
  return count;
}
```

- **Pattern clue:** “Count subarrays with sum = k”
  ✅ → Use **Prefix Sum + HashMap**

⚙️ Prefix Sum Complexity

Operation	Time	Space
Build prefix	O(n)	O(n)
Query sum	O(1)	—

⚙️ 2️⃣ SLIDING WINDOW — Concept

💡 What It Is

The Sliding Window technique keeps a **window (subarray or substring)** that moves across the data to maintain a **running condition**.

📘 Patterns

There are 2 main types:

Type	Example	Window Size
Fixed window	"Find max sum subarray of size k"	Constant size
Variable window	"Find smallest subarray with sum ≥ target"	Expands and shrinks dynamically

---

🧱 Example: Fixed Window

```
int maxSum = 0, windowSum = 0;

for (int i = 0; i < k; i++) windowSum += arr[i];

maxSum = windowSum;


for (int i = k; i < arr.length; i++) {

  windowSum += arr[i] - arr[i - k];
```

```
    maxSum = Math.max(maxSum, windowSum);
}
```

✅ **Pattern clue:** “Subarray of size K with ...” → Use **Fixed Sliding Window**

🧩 **Example: Variable Window**

```
int minLen = Integer.MAX_VALUE, sum = 0;
for (int start = 0, end = 0; end < nums.length; end++) {
    sum += nums[end];
    while (sum >= target) {
        minLen = Math.min(minLen, end - start + 1);
        sum -= nums[start++];
    }
}
```

✅ **Pattern clue:** “Smallest/Largest subarray with sum ≥ / ≤ target” → Use **Variable Sliding Window**

🔥 **When to Use SLIDING WINDOW**

🔍 Problem Type	🔑 Clue Phrases	🚀 Approach
Max/min subarray of size k	“exactly size k”	Fixed window
Longest substring/array without repeat	“longest subarray with condition”	Variable window
Subarray with sum constraint	“sum <, >, ≥, ≤”	Variable window with while loop
Counting distinct elements in window	“number of unique elements in window”	Fixed window + HashMap

🔍 Problem Type	🔑 Clue Phrases	🚀 Approach
Sliding average / temperature	“moving average / continuous range”	Fixed window

---

⚙️ Sliding Window Complexity

Operation	Time	Space
Build window	O(n)	O(1)–O(k)
Move window	O(1) each step	—

🧠 3️⃣ PREFIX SUM vs SLIDING WINDOW — Cheat Comparison

Feature	Prefix Sum	Sliding Window
✅ Best for	Range sum queries, counting subarrays	Finding max/min/longest subarray
💾 Space	O(n)	O(1)
⚡ Speed	Fast for multiple queries	Fast for one-pass problems
🧮 Use case	You *precompute* results	You *adjust window* dynamically
🧩 Example	“Sum between indices L & R?”	“Max sum subarray of size k?”
🧠 Typical pattern	prefix[r] - prefix[l-1]	Expand & shrink pointers

---

💬 4️⃣ Quick Identification Cheat Sheet

🔍 Question Mentions	You Should Think Of
“Find sum between indices / multiple queries”	→ Prefix Sum

🔍 Question Mentions	You Should Think Of
“Count number of subarrays with sum K”	➜ Prefix Sum + HashMap
“Max sum/min sum of size K”	➜ Sliding Window (Fixed)
“Longest substring/array with condition”	➜ Sliding Window (Variable)
“Minimum subarray with sum ≥ target”	➜ Sliding Window (Variable)
“Sum of range [L, R] in grid”	➜ 2D Prefix Sum
“Range update (add X to [L, R])”	➜ Difference Array (prefix trick)

⚙️ Day 1 (Part 3): Sorting + Binary Search

🧭 1️⃣ SORTING ALGORITHMS CHEAT SHEET

Sorting organizes data to make searching, optimization, and pattern detection easier.
There are 3 classic must-know algorithms:

⚡ Quick Sort — “Divide and Conquer on Partition”

Idea:
Pick a *pivot*, partition the array so that smaller elements go left, larger go right.
Then recursively sort both sides.

Code Sketch (Java):

```
void quickSort(int[] arr, int low, int high) {
  if (low < high) {
    int pivot = partition(arr, low, high);
    quickSort(arr, low, pivot - 1);
    quickSort(arr, pivot + 1, high);
  }
```

}

Aspect	Quick Sort
Avg Time	O(n log n)
Worst Time	$O(n^2)$ (if array already sorted, poor pivot)
Space	O(log n) (recursion stack)
Stable	❌ No
Use when	In-place fast sorting is fine (e.g., large unsorted data)

✅ **Pattern Clues:**

- "Need in-place sort"
- "Can rearrange elements freely"
- "Faster than $O(n^2)$ expected"

⚙️ **Merge Sort — "Divide and Conquer with Merge Phase"**

Idea:
Split array in half → recursively sort → merge sorted halves.

Code Sketch (Java):

```
void mergeSort(int[] arr, int l, int r) {
  if (l < r) {
    int mid = (l + r) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    merge(arr, l, mid, r);
  }
}
```

Aspect	Merge Sort
Time	O(n log n) (always)
Space	O(n) (needs temp arrays)
Stable	✅ Yes
Use when	You need guaranteed performance or stable sort (e.g., linked list sort, external sort)

✅ **Pattern Clues:**

- "Large dataset, stability needed"
- "Guaranteed O(n log n)"
- "Sorting linked list"

⚙️ **Counting Sort — "Count Frequencies"**

Idea:
Count occurrences of each value, then reconstruct the array.
Works best for *small integer ranges*.

Code Sketch (Java):

```
int[] countingSort(int[] arr, int maxVal) {
  int[] count = new int[maxVal + 1];
  for (int num : arr) count[num]++;
  int idx = 0;
  for (int i = 0; i <= maxVal; i++) {
    while (count[i]-- > 0) arr[idx++] = i;
  }
  return arr;
}
```

Aspect	Counting Sort
Time	O(n + k) (k = range of numbers)
Space	O(k)
Stable	✅ Yes

Use when Integers, limited range, need linear time

✅ **Pattern Clues:**

- "Numbers within small range (e.g., 0–10000)"
- "Counting frequency / histogram problem"
- "Need to sort non-comparatively"

🔍 **Summary Table**

Algorithm	Time	Space	Stable	Best Use
Quick Sort	O(n log n) avg, O(n^2) worst	O(log n)	❌	In-place sorting
Merge Sort	O(n log n)	O(n)	✅	Stable, predictable sorting
Counting Sort	O(n + k)	O(k)	✅	Small integer range

💬 **When You Hear in Questions...**

Question Phrase	Likely Algorithm
"In-place fast sorting, average O(n log n)"	→ Quick Sort
"Stable guaranteed O(n log n)"	→ Merge Sort
"Integers in small range / counting elements"	→ Counting Sort
"Need to sort linked list"	→ Merge Sort (no random access)
"Sorting for frequency or bucket grouping"	→ Counting Sort / Bucket Sort

---

🧠 2️⃣ BINARY SEARCH & ITS PATTERNS

Binary Search is **not just for searching numbers** — it's a *pattern* used anywhere you can "decide which half to discard"
(i.e., a **monotonic** condition: true → false or false → true transition).

⚙️ Classic Binary Search

Find element in sorted array

```
int binarySearch(int[] arr, int target) {
  int low = 0, high = arr.length - 1;
  while (low <= high) {
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) low = mid + 1;
    else high = mid - 1;
  }
  return -1;
}
```

Time: O(log n)
Space: O(1)

🧩 Binary Search Patterns Cheat Sheet

🔍 Pattern	Description	Example Problem	Template Hint
1. Classic Search	Find element in sorted array	Binary Search (LC 704)	while (low <= high)

🔍 Pattern	Description	Example Problem	Template Hint
2. First / Last Occurrence	Find leftmost/rightmost index	LC 34 – First and Last Position	Use modified condition (mid boundary check)
3. Search Insert Position	Where to insert to keep sorted	LC 35	Same as search, return low
4. Binary Search on Answer (Parametric Search)	Search on numeric result space, not array	LC 875 – Koko Eating Bananas	while (low < high) and check mid feasibility
5. Binary Search in Rotated Array	Search in rotated sorted array	LC 33, LC 81	Check which half is sorted
6. Peak / Minimum Element Search	Find local max/min	LC 162 – Peak Element	Compare mid and mid+1
7. Square Root / Precision Search	Binary search on continuous range	LC 69 – Sqrt(x)	Adjust precision condition
8. 2D Matrix Search	Treat matrix as sorted array	LC 74	Flatten indices or row-column elimination

---

🧱 Template 1 — Classic Integer Binary Search

```
while (low <= high) {
   int mid = low + (high - low) / 2;
   if (condition(mid))
      high = mid - 1;  // move left
   else
      low = mid + 1;   // move right
}
```

🧱 Template 2 — Binary Search on Answer (Most Common Trick)

Used when the *answer itself* lies between [low, high], and we can test if a value is feasible.

Example — Koko Eating Bananas 🍌
Find smallest speed so Koko finishes in h hours.

```
int low = 1, high = max(piles);

while (low < high) {

   int mid = (low + high) / 2;

   if (canEatAll(mid)) high = mid;

   else low = mid + 1;

}

return low;
```

✅ Pattern clue:

“Find minimum/maximum X such that condition is satisfied”
→ Use **Binary Search on Answer**

🔍 Quick Recognition Table

Question Mentions	Use Binary Search Pattern
“Sorted array”	Classic search
“Find first / last position”	Boundary binary search
“Find min/max that satisfies condition”	Binary search on answer
“Monotonic condition (e.g., can/cannot, fits/doesn’t fit)”	Binary search on answer
“Rotated sorted array”	Modified binary search
“Search in 2D matrix”	Flattened binary search

🧠 Binary Search Time-Space

Operation	Time	Space
Basic search	O(log n)	O(1)
On answer space	O(log(range))	O(1)

💭 **Summary Mindset**

If question says...	Think...
“Need to sort quickly”	➜ Quick Sort
“Stable guaranteed sorting”	➜ Merge Sort
“Integer range known”	➜ Counting Sort
“Find min/max possible value under condition”	➜ Binary Search on Answer
“Search in sorted structure”	➜ Binary Search
“Array rotated / peaks / position-based search”	➜ Binary Search variant

⚙️ **Day 2 (Part 1): HashMap / HashSet Patterns + Prefix Hashing**

🧭 1️⃣ **Core Idea — Why HashMap / HashSet?**

💡 **Concept:**

HashMap and HashSet are **O(1)** average-time tools for lookups and frequency tracking. They are your **go-to data structures** for:

- Detecting duplicates
- Counting occurrences
- Tracking seen elements
- Storing prefix sums/frequencies

🧩 2️⃣ **Difference Between HashMap vs HashSet**

Feature	HashMap	HashSet
Stores	Key → Value pairs	Unique Keys only
Access time	O(1) average	O(1) average
Use for	Counting frequency, mapping relations, prefix sums	Detecting duplicates or existence
Example	{2→3, 5→1} → element counts	{2,5,9} → seen numbers

⚡ 3️⃣ Pattern Recognition — When to Use

Problem Type	🔑 Clue Phrases	🚀 Approach
Duplicates check	“Find if array has duplicates”	Use HashSet to track seen elements
Pair matching (Two Sum)	“Find two numbers that sum to K”	Use HashMap to store complements
Subarray tracking (Sum = K)	“Count subarrays with given sum”	Prefix Sum + HashMap of prefix frequencies
Frequency counting	“Find element with max frequency”	Use HashMap<Integer,Integer>
Anagrams / Group by pattern	“Group similar strings by rearrangement”	Use HashMap<String,List<String>>
Prefix hashing	“Compare substrings quickly or find repeated substring”	Store prefix hashes / prefix sums

---

🧮 4️⃣ Pattern 1 — Two Sum

🧠 **Problem Statement:**

Find indices of two numbers that add up to a target k.

Example:
arr = [2,7,11,15], target = 9 → [0,1]

💡 Approach:

Store numbers in HashMap as you iterate.
At each step, check if (target - num) already exists.

Code:

```
int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement))
      return new int[]{map.get(complement), i};
    map.put(nums[i], i);
  }
  return new int[]{-1, -1};
}
```

Step Explanation

1️⃣ Iterate elements

2️⃣ For each, compute complement = target - current

3️⃣ If complement in map → found pair

4️⃣ Otherwise store current in map

✅ Pattern Clues:

- "Find two elements that sum/multiply to target"
- "Return indices or count pairs"
  ➞ Use HashMap (complement storage)

Time: O(n)
Space: O(n)

🧩 5️⃣ Pattern 2 — Subarray Sum = K

🧠 Problem Statement:

Count subarrays whose **sum = K**.

Example:
nums = [1, 2, 3], k = 3 → 2
(Subarrays: [1,2], [3])

💡 Approach:

Use **Prefix Sum + HashMap**
Store how many times each prefix sum has occurred.

Code:

```
int subarraySum(int[] nums, int k) {
   Map<Integer, Integer> map = new HashMap<>();
   map.put(0, 1); // base case: sum=0 appears once
   int sum = 0, count = 0;

   for (int num : nums) {
      sum += num;
      if (map.containsKey(sum - k))
         count += map.get(sum - k);
      map.put(sum, map.getOrDefault(sum, 0) + 1);
   }
   return count;
}
```

Step Explanation

1️⃣ Keep running prefix sum

2️⃣ Check if (sum - k) exists → means subarray ending here = k

3️⃣ Update map with current prefix count

✅ **Pattern Clues:**

- "Count subarrays / continuous sequence with sum = target"
- "Array contains +ve / -ve numbers"
  ➞ **Use Prefix Sum + HashMap (frequency)**

Time: O(n)
Space: O(n)

⚙️ 6️⃣ **Pattern 3 — Prefix Hashing**

Prefix Hashing is an **extension of prefix sum**, but instead of sums, you store **computed hashes or encoded values** of prefixes to compare substrings efficiently.

💡 **Use Cases**

Problem Type	Example	Approach
Compare substrings quickly	"Check if two substrings are equal"	Hash each prefix and compare hash differences
Detect repeated substrings	"Longest duplicate substring"	Use Rolling Hash + Binary Search
Subarray balance	"Equal 0s and 1s in binary array"	Convert 0→-1 and use Prefix Sum + HashMap
Pattern finding	"Equal prefix-suffix"	Compare hash values or prefix arrays

---

🧠 **Example: Equal 0s and 1s**

"Find longest subarray with equal 0s and 1s"

Convert 0 → -1
Then it becomes “longest subarray with sum = 0”.

Code:

```
int findMaxLength(int[] nums) {
  Map<Integer, Integer> map = new HashMap<>();
  map.put(0, -1);
  int sum = 0, maxLen = 0;
  for (int i = 0; i < nums.length; i++) {
    sum += (nums[i] == 0 ? -1 : 1);
    if (map.containsKey(sum))
      maxLen = Math.max(maxLen, i - map.get(sum));
    else
      map.put(sum, i);
  }
  return maxLen;
}
```

✅ **Pattern Clues:**

- “Equal number of X and Y”
- “Longest subarray with balanced values”
  ➞ Prefix Sum / HashMap + Hash Encoding pattern

🧩 7️⃣ **Recognition Summary Table**

🔍 Question Mentions	🧠 Pattern to Use	💾 Data Structure
“Check duplicates / seen elements”	Membership check	HashSet

🔍 Question Mentions	🧠 Pattern to Use	💾 Data Structure
“Find two numbers that sum to K”	Store complement	HashMap<num, index>
“Count subarrays with sum = K”	Prefix Sum + Frequency	HashMap<sum, count>
“Find longest balanced subarray”	Prefix Sum + First Occurrence	HashMap<sum, index>
“Compare or hash substrings”	Prefix Hashing	Prefix array
“Group anagrams”	Key by signature	HashMap<String, List<String>>

⚡ 8️⃣ Complexity Overview

Operation	Time	Space
Insert / Lookup in HashMap	O(1) avg	O(n)
Two Sum	O(n)	O(n)
Subarray Sum = K	O(n)	O(n)
Prefix Hashing	O(n)	O(n)

🧩 9️⃣ Practice Problems

Topic	Problem	Difficulty
Two Sum	LC 1 - Two Sum	Easy
Subarray Sum = K	LC 560 - Subarray Sum Equals K	Medium
Equal 0s and 1s	LC 525 - Contiguous Array	Medium
Longest Substring w/o Repeat	LC 3 - Longest Substring Without Repeating Characters	Medium

Topic	Problem	Difficulty
Group Anagrams	LC 49 - Group Anagrams	Medium

🎯 1 0 Quick Mental Map (for interviews)

If question says...	Think of...	Why
“Find pair with given sum”	➜ Two Sum	Complement logic
“Count subarrays = K”	➜ Prefix Sum + HashMap	Prefix frequency
“Find subarray with equal 0s & 1s”	➜ Prefix Hashing	0→-1 + prefix balance
“Detect duplicates”	➜ HashSet	O(1) lookup
“Track frequency”	➜ HashMap	Counting
“String grouping / pattern mapping”	➜ HashMap	Key by pattern

Perfect 🔥 — we’re entering **Day 3 (String + Sliding Window Mastery)**.
This day connects **string problems** with **sliding window**, **hashmaps**, and **pattern searching** — a super common area for interviews (especially for FAANG / product companies).

Let’s build your **cheat sheet + pattern recognition map** just like before 👇

🧩 Day 3: String Manipulation + Sliding Window + KMP / Rabin-Karp

---

🧭 1 Core Concepts

Concept	What It’s For	Common Questions
String manipulation	Reversing, rotating, substring ops	Palindrome check, Anagram grouping

Concept	What It's For	Common Questions
Sliding Window	Fixed / dynamic range substring problems	Longest substring, Anagrams in string
KMP Algorithm	Fast pattern search (O(n))	"Find pattern in text"
Rabin–Karp	Rolling hash–based pattern matching	"Find substring occurrences efficiently"

⚡ 2️⃣ Sliding Window Refresher (Applied to Strings)

Think of **sliding window** as maintaining a **substring range [L...R]** that satisfies a property. You **move right (expand)** and **move left (shrink)** dynamically.

✨ When to Use Sliding Window

Clues in Question	Use Sliding Window?
"Longest / shortest substring that..."	✅ YES
"Substring without repeating characters"	✅ YES
"Find all substrings of length K with..."	✅ YES
"Number of substrings that..."	✅ YES
"Minimum window substring containing pattern"	✅ YES

⚙️ 3️⃣ Basic Template — Sliding Window (Variable Size)

```
int left = 0;

for (int right = 0; right < s.length(); right++) {

  // Expand window

  // Update counts / check condition


  while (/* condition invalid */) {
```

```
    // Shrink window
    left++;
  }

  // Update answer if valid window
}
```

✅ **Key Trick**:
Maintain frequency map or counter of what's inside the window.
This helps detect "valid window" conditions.

🧩 🔠 Pattern 1 — Anagram Detection in String

🧠 Problem:

Find all starting indices of p's anagrams in s.
Input: s = "cbaebabacd", p = "abc"
Output: [0, 6]

💡 Approach: Sliding Window + Frequency Matching

- Keep frequency of p (target)
- Use window of size p.length()
- Compare frequency with current substring

Code:

```
List<Integer> findAnagrams(String s, String p) {
  List<Integer> res = new ArrayList<>();
  if (s.length() < p.length()) return res;

  int[] target = new int[26];
  int[] window = new int[26];
```

```
    for (char c : p.toCharArray()) target[c - 'a']++;

    for (int i = 0; i < s.length(); i++) {
        window[s.charAt(i) - 'a']++;
        if (i >= p.length())
            window[s.charAt(i - p.length()) - 'a']--;
        if (Arrays.equals(target, window))
            res.add(i - p.length() + 1);
    }
    return res;
}
```

Pattern	**Sliding window (fixed size)**
Window size	length of pattern
Condition	Equal frequency count
Time	O(n)
Space	O(26) = O(1)

✅ **When to use:**

“Find all permutations / anagrams of a pattern in a string”
“Substring has same character multiset”

🧩 5️⃣ **Pattern 2 — Palindrome Check / Longest Palindrome**

💡 1️⃣ **Simple Palindrome Check**

```
boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j)
```

```
    if (s.charAt(i++) != s.charAt(j--))
        return false;
  return true;
}
```

💡 2️⃣ Longest Palindromic Substring (Expand Around Center)

```
String longestPalindrome(String s) {
  int start = 0, end = 0;
  for (int i = 0; i < s.length(); i++) {
    int len1 = expand(s, i, i);      // odd
    int len2 = expand(s, i, i + 1);   // even
    int len = Math.max(len1, len2);
    if (len > end - start) {
      start = i - (len - 1) / 2;
      end = i + len / 2;
    }
  }
  return s.substring(start, end + 1);
}

int expand(String s, int l, int r) {
  while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) {
    l--; r++;
  }
  return r - l - 1;
}
```

✅ **Pattern clues:**

- "Palindrome" → Think **expand around center**
- "Longest palindromic substring" → **O(n^2)** expand approach

---

⚙️ 6️⃣ **Pattern 3 — Longest Substring Without Repeating Characters**

💡 **Problem:**

Find length of longest substring without repeating characters.
s = "abcabcbb" → 3

Approach: Sliding window + HashSet

```
int lengthOfLongestSubstring(String s) {
  Set<Character> set = new HashSet<>();
  int left = 0, maxLen = 0;

  for (int right = 0; right < s.length(); right++) {
    while (set.contains(s.charAt(right)))
      set.remove(s.charAt(left++));
    set.add(s.charAt(right));
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
```

✅ **Pattern Clues:**

- "Without repeating characters"
- "Distinct substring"
  ➞ **Use Sliding Window + HashSet**

Time: O(n)
Space: O(n)

🧩 7️⃣ Pattern 4 — KMP Algorithm (Knuth–Morris–Pratt)

💡 Use Case

Efficient pattern search in text (O(n+m))

Instead of re-checking from scratch after mismatch,
KMP uses a **"LPS (Longest Prefix Suffix)"** table to skip comparisons.

🔹 Steps

1. Build **LPS array** for pattern.
    - lps[i] = longest proper prefix that is also a suffix.
2. Traverse text using this pattern knowledge.

🔹 Example:

Pattern: "ABABCABAB"

i	pattern[i]	lps[i]
0	A	0
1	B	0
2	A	1
3	B	2
4	C	0
5	A	1
6	B	2
7	A	3

i	pattern[i]	lps[i]
8	B	4

- **Code (Search)**

```
List<Integer> KMPSearch(String pat, String txt) {
   int M = pat.length(), N = txt.length();
   int[] lps = buildLPS(pat);
   List<Integer> res = new ArrayList<>();
   int i = 0, j = 0;

   while (i < N) {
      if (pat.charAt(j) == txt.charAt(i)) { i++; j++; }
      if (j == M) {
         res.add(i - j);
         j = lps[j - 1];
      } else if (i < N && pat.charAt(j) != txt.charAt(i)) {
         if (j != 0) j = lps[j - 1];
         else i++;
      }
   }
   return res;
}

int[] buildLPS(String pat) {
   int[] lps = new int[pat.length()];
```

```
  for (int i = 1, len = 0; i < pat.length();) {
    if (pat.charAt(i) == pat.charAt(len)) {
      lps[i++] = ++len;
    } else if (len != 0) len = lps[len - 1];
    else lps[i++] = 0;
  }
  return lps;
}
```

✅ **Pattern Clues:**

- "Find pattern in text"
- "Search substring efficiently"
  ➞ **Use KMP (O(n))**

🧩 8️⃣ **Pattern 5 — Rabin-Karp (Rolling Hash)**

💡 **Idea:**

Convert substring → **numeric hash**,
then compare hashes instead of characters.

🔹 **Core Formula**

hash(s[0..m-1]) = (s[0]*p^(m-1) + s[1]*p^(m-2) + ... + s[m-1]) % MOD

For next substring, reuse previous hash by removing first char and adding new one — this gives **O(1) per step**.

🔹 **Pros & Cons**

Advantage	**Limitation**
Fast average	Hash collision possible
Easy rolling hash	Needs modulo handling

✅ **Pattern Clues:**

- “Search substring efficiently”
- “Detect plagiarism / repeated substring”
  ➞ **Use Rabin–Karp**

🧮 🔟 Recognition Table

🔍 Problem Mentions	🚀 Pattern	🔧 Technique
“Longest substring / smallest window”	Sliding Window	Expand / Shrink dynamically
“Substring with no repeats”	Sliding Window + HashSet	
“Find all anagrams of pattern”	Sliding Window + Frequency	
“Check palindrome”	Two-pointer	
“Longest palindromic substring”	Expand from center	
“Search substring pattern”	KMP / Rabin-Karp	
“Repeated substring check”	Prefix Hash / Rabin-Karp	

🧠 1 0 Practice Problems

Topic	Problem	Difficulty
Longest Substring w/o Repeat	LC 3	Medium
Find All Anagrams	LC 438	Medium
Minimum Window Substring	LC 76	Hard
Longest Palindromic Substring	LC 5	Medium
Implement strStr (KMP)	LC 28	Easy
Repeated Substring Pattern	LC 459	Easy

🧭 Quick "Pattern-Spotting" Cheat Sheet

If question says...	Think...	Technique
"Substring of length k / smallest window"	🔹 Sliding Window	Move left/right pointers
"Without repeating chars"	🔹 Sliding Window + Set	Track seen chars
"Permutation / Anagram check"	🔹 Sliding Window + Count Array	Compare frequencies
"Palindrome"	🔹 Two-pointer / Expand around center	
"Pattern search in text"	🔹 KMP	Prefix table
"Fast substring match (hash)"	🔹 Rabin–Karp	Rolling hash

🧩 Day 4: Linked List (Single, Double, Fast/Slow Pointer Patterns)

⚙️ 1️⃣ Core Concepts

Concept	Key Idea	When to Use
Singly Linked List	Nodes with value and next pointer	Traversal / reversal / merging
Doubly Linked List	Nodes with prev, next pointers	LRU Cache, undo/redo
Fast/Slow Pointer	Two pointers moving at different speeds	Cycle detection, middle element
Dummy Node Trick	Use a fake head to simplify edge cases	Merging, deletion, reversal

---

🧭 2️⃣ Basic Structures (Java)

```
class ListNode {
   int val;
   ListNode next;
   ListNode(int val) { this.val = val; }
}

class DNode {
   int val;
   DNode prev, next;
   DNode(int val) { this.val = val; }
}
```

🔧 3️⃣ Common Linked List Operations

Operation	Code	Time
Insert at head	node.next = head; head = node;	O(1)
Insert at tail	Traverse → tail.next = node;	O(n)
Delete node	prev.next = node.next;	O(1) (if prev known)
Search	Traverse till found	O(n)
Reverse	Pointer manipulation	O(n)

🔹 Reverse Linked List (Iterative)

```
ListNode reverse(ListNode head) {
   ListNode prev = null, curr = head;
   while (curr != null) {
      ListNode next = curr.next;
```

```
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Steps:

1. Save next
2. Reverse curr.next
3. Move forward

✅ **Pattern Clues:**

- "Reverse the list"
- "Reverse k nodes at a time"
  → Use **3-pointer reversal pattern**

🔹 **Reverse Linked List (Recursive)**

```
ListNode reverseRec(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode rest = reverseRec(head.next);
    head.next.next = head;
    head.next = null;
    return rest;
}
```

✅ Elegant but slightly more memory (recursion stack)

⚡ 🔼 **Fast & Slow Pointer — Concept**

Two pointers moving at different speeds.
slow = slow.next; fast = fast.next.next;

💡 Use Cases

Problem	Trick	Why
Find middle node	fast moves 2× faster	When fast hits end → slow = middle
Detect cycle	Fast meets slow if loop exists	Floyd's algorithm
Find start of cycle	Reset slow to head after meeting	Works due to distance math
Intersection of two lists	Equalize lengths, move together	Same node → intersection

---

🧩 5️⃣ Pattern 1 — Find Middle of Linked List

```
ListNode middleNode(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow;
}
```

✅ **Clues:**

- "Find middle node"
- "Split linked list in half"
  ➞ Use **fast/slow pointers**

🧩 6️⃣ Pattern 2 — Detect Cycle (Floyd's Cycle Detection)

```
boolean hasCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow == fast) return true;
  }
  return false;
}
```

✅ **Pattern Clues:**

- "Check if list has a loop"
- "Detect circular dependency"
  ➞ Use **fast/slow pointers**

Time: O(n)
Space: O(1)

🧩 7️⃣ Pattern 3 — Find Start of Cycle

After detecting meeting point in loop:

- Move slow to head.
- Move both slow and fast one step each.
- When they meet → start of cycle.

```
ListNode detectCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
```

```
      fast = fast.next.next;
      if (slow == fast) {
        slow = head;
        while (slow != fast) {
          slow = slow.next;
          fast = fast.next;
        }
        return slow; // cycle start
      }
    }
    return null;
}
```

✅ **Pattern Clues:**

- "Find start node of loop"
- "Linked list loops back to earlier node"
  ➞ **Floyd's Algorithm (Phase 2)**

🧩 8️⃣ **Pattern 4 — Intersection of Two Linked Lists**

Given headA and headB, find node where they intersect (by reference).

💡 **Approach 1 — Length Difference Trick**

1. Find lengths of A, B.
2. Advance longer list by |lenA - lenB|.
3. Move both together until they meet.

```
ListNode getIntersection(ListNode a, ListNode b) {
  int lenA = length(a), lenB = length(b);
```

```
    while (lenA > lenB) { a = a.next; lenA--; }
    while (lenB > lenA) { b = b.next; lenB--; }
    while (a != b) { a = a.next; b = b.next; }
    return a;
}

int length(ListNode node) {
    int len = 0;
    while (node != null) { len++; node = node.next; }
    return len;
}
```

💡 **Approach 2 — Pointer Switch Trick (Elegant)**

Traverse both lists.
When one reaches null, switch it to the other's head.
They'll meet at intersection or both null.

```
ListNode getIntersectionNode(ListNode a, ListNode b) {
    if (a == null || b == null) return null;
    ListNode p1 = a, p2 = b;
    while (p1 != p2) {
        p1 = (p1 == null) ? b : p1.next;
        p2 = (p2 == null) ? a : p2.next;
    }
    return p1;
}
```

✅ **Pattern Clues:**

- “Find intersection node (by reference, not value)”
- “Two linked lists merge like Y shape”
  ➞ **Use length equalization or pointer switch**

🧩 🪪 Pattern 5 — Reverse in Groups of K

Reverse every k nodes (common Amazon/Google pattern)

```
ListNode reverseKGroup(ListNode head, int k) {
  ListNode curr = head;
  int count = 0;
  while (curr != null && count < k) {
    curr = curr.next;
    count++;
  }
  if (count == k) {
    curr = reverseKGroup(curr, k);
    while (count-- > 0) {
      ListNode temp = head.next;
      head.next = curr;
      curr = head;
      head = temp;
    }
    head = curr;
  }
  return head;
}
```

✅ **Pattern Clues:**

- “Reverse nodes in groups”
- “Reverse alternate k nodes”
  ➞ **Recursive or iterative reversal in chunks**

🧩 🔁 1 0 Doubly Linked List — Quick Notes

Operation	Code / Concept
Insert after node	Update next & prev links both sides
Delete node	node.prev.next = node.next; node.next.prev = node.prev;
Use case	LRU Cache, Undo/Redo, Browser history

✅ **Interview Tip:** Doubly linked list is rarely asked directly; usually as **part of design problem (LRU Cache)**.

🧠 1 1 Recognition Summary

🔍 Question Mentions	🚀 Pattern	🔧 Technique
“Find middle node”	Fast/slow pointer	Move fast 2×
“Detect cycle”	Floyd’s Cycle Detection	Fast/slow
“Find cycle start”	Reset slow after meeting	Fast/slow
“Reverse list”	3-pointer	Iterative / Recursive
“Reverse in groups”	K reversal	Recursion
“Intersection node”	Pointer switching	Equalize lengths
“Implement LRU”	Doubly Linked List + HashMap	

📚 1 2 Practice Problems

Topic	Problem	Difficulty
Reverse Linked List	LC 206	Easy
Middle of Linked List	LC 876	Easy
Linked List Cycle	LC 141	Easy
Linked List Cycle II	LC 142	Medium
Intersection of Linked Lists	LC 160	Easy
Reverse Nodes in k-Group	LC 25	Hard
LRU Cache	LC 146	Medium

🧭 1️⃣3️⃣ Quick "Pattern-Spotting" Table

If Question Says...	Think of...	Why
"Find middle"	Fast/Slow	Fast = 2× speed
"Loop / repeated node"	Cycle Detection	Floyd's algo
"Where loop begins"	Cycle Start	Reset slow
"Reverse list"	Pointer reversal	
"Merge / split lists"	Dummy node	Simplifies edge cases
"Find intersection"	Pointer switch	Elegant O(1) space
"LRU / browser history"	Doubly Linked List	Back/forward navigation

---

🧭 Day 6 – Stacks & Queues (Cheat Sheet + Pattern Guide)

🧱 1️⃣ Stack Basics

Definition:
A **LIFO (Last In, First Out)** data structure — the last inserted element is the first removed.

🧩 Implementation

```
// Using Array
Stack<Integer> st = new Stack<>();

// Using LinkedList (manual)
class Node { int val; Node next; Node(int v){ val=v; } }
class StackLL {
  Node top;
  void push(int x){ Node n=new Node(x); n.next=top; top=n; }
  int pop(){ int v=top.val; top=top.next; return v; }
}
```

📚 2️⃣ Queue Basics

Definition:
A **FIFO (First In, First Out)** structure — elements are processed in order of arrival.

🧩 Implementation

```
Queue<Integer> q = new LinkedList<>();

// Manual array-based circular queue
class QueueArray {
  int[] arr; int front, rear, size;
  QueueArray(int cap){ arr=new int[cap]; }
  void enqueue(int x){ arr[rear++ % arr.length]=x; size++; }
  int dequeue(){ int val=arr[front++ % arr.length]; size--; return val; }
}
```

🧠 3️⃣ When to Use

Pattern	When to Think of It	Typical Questions
Stack	Backtracking, undo, nested structure, last-seen pattern	Valid parentheses, next greater element, Min Stack
Queue	Order-based traversal, level order BFS, producer-consumer	BFS in graph/tree, sliding window max
Deque	Need access to both ends	Sliding window max/min

⚡ 4️⃣ Monotonic Stack (🔥 Interview Favorite)

Idea:
Maintain elements in **sorted order (increasing or decreasing)** inside a stack while scanning.

Useful for:

- Finding **Next Greater / Smaller** element
- Finding **Span / Stock span problem**
- Histogram area problems

🧩 Pattern Template (Next Greater Element)

```
int[] nextGreater(int[] arr) {
  Stack<Integer> st = new Stack<>();
  int[] res = new int[arr.length];
  Arrays.fill(res, -1);

  for (int i = 0; i < arr.length; i++) {
    while (!st.isEmpty() && arr[i] > arr[st.peek()]) {
      res[st.pop()] = arr[i];
```

```
    }
    st.push(i);
  }
  return res;
}
```

🧠 **When to use:**
If the question says:

"For each element, find the next greater/smaller element"
or
"Find the next warmer day"
→ Immediately think **Monotonic Stack**

🧱 **Example: Largest Rectangle in Histogram**

- Maintain **increasing stack**
- When current height < stack top → pop and calculate area
  → **Pattern keyword:** "Rectangle / Max Area / Histogram"

🧭 5️⃣ **Min Stack**

Goal: Retrieve min element in O(1) time.

```
class MinStack {
  Stack<Integer> st = new Stack<>();
  Stack<Integer> min = new Stack<>();

  void push(int x){
    st.push(x);
    if(min.isEmpty() || x <= min.peek()) min.push(x);
  }
```

```
  void pop(){
    if(st.pop().equals(min.peek())) min.pop();
  }

  int getMin(){ return min.peek(); }
}
```

🧠 **When to use:**
If asked:

“Design a stack that supports push/pop/getMin in O(1)”
→ Instantly think **Two Stacks** (main + min)

⚙️ 6️⃣ **Valid Parentheses Pattern**

Use Stack to match pairs:

```
boolean isValid(String s){
  Stack<Character> st = new Stack<>();
  for(char c : s.toCharArray()){
    if(c=='(') st.push(')');
    else if(c=='{') st.push('}');
    else if(c=='[') st.push(']');
    else if(st.isEmpty() || st.pop()!=c) return false;
  }
  return st.isEmpty();
}
```

🧠 **When to use:**
If you see words like:

“Balanced”, “Nested”, “Valid expression”
→ **Stack pattern**

🚀 7️⃣ Next Greater Element (NGE)

Already seen above — but this concept applies in **arrays and circular arrays.**

For circular:

```
int n = arr.length;

for (int i = 0; i < 2 * n; i++) {

  while (!st.isEmpty() && arr[i % n] > arr[st.peek()])

    res[st.pop()] = arr[i % n];

  if (i < n) st.push(i);

}
```

🧠 **When to use:**

“Find element’s next greater on right side”
or
“Temperatures problem / Circular array relationship”
→ **Monotonic stack**

🧩 8️⃣ Queue-Based Patterns

🔹 BFS / Level Order

Always use Queue.

```
Queue<TreeNode> q = new LinkedList<>();

q.offer(root);

while(!q.isEmpty()){

  TreeNode cur = q.poll();

  if(cur.left!=null) q.offer(cur.left);

  if(cur.right!=null) q.offer(cur.right);
```

}

🔹 **Sliding Window Maximum**

Use **Deque** to maintain decreasing order of elements inside the current window.

🧭 9️⃣ **Pattern Recognition Table**

Keyword in Question	Pattern / DS to Think
“Balanced”, “Valid expression”, “Next greater/smaller”, “Undo”, “Nested”	**Stack**
“FIFO order”, “Level traversal”, “Producer-consumer”, “Sliding window”	**Queue / Deque**
“Rectangle area”, “Stock span”, “Warmer day”	**Monotonic Stack**
“Min in constant time”	**Min Stack**

🧠 1️⃣0️⃣ **Practice Problems (in order)**

Problem	Type
Valid Parentheses	Stack
Min Stack	Stack
Next Greater Element	Monotonic Stack
Daily Temperatures	Monotonic Stack
Largest Rectangle in Histogram	Monotonic Stack
Sliding Window Maximum	Deque
Implement Stack using Queues	Queue
Implement Queue using Stacks	Stack

Awesome ⚡ You're now at **Day 7 — Graphs**, one of the most conceptually deep yet *pattern-rich* DSA topics.
Below is your **complete "Graph Cheat Sheet" + pattern guide** — structured for interviews so you can quickly recognize which algorithm to use from the question keywords.

🧭 Day 7 – Graphs (Representation, Traversals & Core Algorithms)

🧩 1️⃣ Graph Representation

Graphs can be represented in 2 major ways:

Representation	Description	Space	When to Use
Adjacency Matrix	2D array n x n, where matrix[i][j] = 1 if edge exists	$O(V^2)$	Dense graphs (many edges)
Adjacency List	Array/List of lists where each index stores connected nodes	O(V + E)	Sparse graphs (fewer edges)

Example — Adjacency List

```
int V = 5;

List<List<Integer>> graph = new ArrayList<>();

for (int i = 0; i < V; i++) graph.add(new ArrayList<>());


graph.get(0).add(1);

graph.get(0).add(2);

graph.get(1).add(3);

graph.get(2).add(4);
```

Visual:

0 -> 1, 2

1 -> 3

2 -> 4

3 ->

4 ->

Example — Adjacency Matrix

int[][] matrix = new int[V][V];

matrix[0][1] = 1;

matrix[0][2] = 1;

matrix[1][3] = 1;

matrix[2][4] = 1;

🧠 2️⃣ When to Use What

Keyword in Question	You should think
“Find connected components”, “Check if path exists”	BFS / DFS
“Order of tasks”, “Dependency resolution”	Topological Sort
“Find shortest distance / minimum cost”	Dijkstra or BFS (if unweighted)
“Cycle detection / grouping nodes”	Union-Find
“Minimum Spanning Tree”	Kruskal / Prim
“Find if graph is bipartite”	BFS with coloring

⚡ 3️⃣ BFS (Breadth-First Search)

Idea:
Visit neighbors **level by level** using a **Queue**.

void bfs(List<List<Integer>> graph, int start) {

```
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;

    while (!q.isEmpty()) {
        int node = q.poll();
        for (int nbr : graph.get(node)) {
            if (!visited[nbr]) {
                visited[nbr] = true;
                q.offer(nbr);
            }
        }
    }
}
```

🧠 **When to use:**

- Shortest path in **unweighted** graphs
- Level-order traversal
- Check **connected components**

🧩 **Pattern Keyword Triggers:**

“Minimum steps”, “Fewest moves”, “Level order”, “Distance = K” → **BFS**

⚙️ 🟦 **DFS (Depth-First Search)**

Idea:
Go **as deep as possible**, then backtrack.

```
void dfs(List<List<Integer>> graph, int node, boolean[] visited) {
```

```
  visited[node] = true;
  for (int nbr : graph.get(node)) {
    if (!visited[nbr]) dfs(graph, nbr, visited);
  }
}
```

🧠 **When to use:**

- Detect cycle
- Find connected components
- Topological sort
- Solve backtracking in graph form (e.g., islands, path finding)

🧩 **Pattern Keywords:**

"All possible paths", "Check if cycle exists", "Explore islands" → **DFS**

🧩 5️⃣ **Topological Sort (Directed Acyclic Graphs Only)**

Definition:
Linear ordering of vertices such that for every directed edge (u → v), **u comes before v**.

Approach 1 — DFS + Stack

```
void topoDFS(int node, boolean[] visited, Stack<Integer> st, List<List<Integer>> graph){
  visited[node] = true;
  for(int nbr : graph.get(node)){
    if(!visited[nbr]) topoDFS(nbr, visited, st, graph);
  }
  st.push(node);
}
```

Approach 2 — Kahn's Algorithm (BFS + In-degree)

```
Queue<Integer> q = new LinkedList<>();
```

```
int[] indeg = new int[V];
for (int u = 0; u < V; u++)
   for (int v : graph.get(u)) indeg[v]++;

for (int i = 0; i < V; i++) if (indeg[i] == 0) q.offer(i);

while(!q.isEmpty()){
   int node = q.poll();
   for(int nbr : graph.get(node)){
      if(--indeg[nbr]==0) q.offer(nbr);
   }
}
```

🧠 **When to use:**

“Course schedule”, “Dependency order”, “Build system” → **Topological Sort**

🧩 6️⃣ **Union-Find (Disjoint Set Union – DSU)**

Used for:
Detecting cycles, grouping connected components, or merging sets (in Kruskal’s MST, etc.)

Code Snippet

```
int[] parent, rank;

int find(int x){
   if(parent[x]!=x) parent[x] = find(parent[x]); // Path compression
   return parent[x];
}
```

```
void union(int x, int y){
   int px = find(x), py = find(y);
   if(px==py) return;
   if(rank[px] < rank[py]) parent[px]=py;
   else if(rank[px] > rank[py]) parent[py]=px;
   else { parent[py]=px; rank[px]++; }
}
```

🧠 **When to use:**

- Detect cycle in undirected graph
- Kruskal's algorithm
- "Are nodes connected?" type queries

🧩 **Pattern Keywords:**

"Union", "Find", "Connected components", "Cycle detection", "Grouping" → **Union-Find**

🚀 7️⃣ **Shortest Path — Dijkstra's Algorithm**

Used for:
Weighted graphs (no negative weights)

Idea:
Use a **PriorityQueue (Min-Heap)** to pick the next node with the smallest distance.

```
class Pair { int node, dist; Pair(int n, int d){ node=n; dist=d; } }


int[] dijkstra(int V, List<List<Pair>> graph, int src){
   int[] dist = new int[V];
   Arrays.fill(dist, Integer.MAX_VALUE);
   dist[src] = 0;
   PriorityQueue<Pair> pq = new PriorityQueue<>((a,b)->a.dist-b.dist);
```

```
    pq.offer(new Pair(src, 0));

    while(!pq.isEmpty()){
        Pair cur = pq.poll();
        for(Pair edge : graph.get(cur.node)){
            if(dist[cur.node] + edge.dist < dist[edge.node]){
                dist[edge.node] = dist[cur.node] + edge.dist;
                pq.offer(new Pair(edge.node, dist[edge.node]));
            }
        }
    }
    return dist;
}
```

🧠 **When to use:**

“Shortest path with weights”, “Minimum cost”, “Network latency”, “Map routing” → **Dijkstra**

📋 🎱 **Pattern Recognition Table**

Keyword / Phrase in Question	Think Algorithm
"Find all reachable nodes", "Connected components", "Traversal"	BFS / DFS
"Fewest steps / levels / min moves"	BFS
"All paths / recursive exploration"	DFS
"Dependency order / course schedule"	Topological Sort
"Are A and B connected?", "Detect cycle"	Union-Find
"Shortest weighted path"	Dijkstra

Keyword / Phrase in Question	Think Algorithm
"Unweighted shortest path"	BFS

🧩 9️⃣ Visual Summary

Graph Algorithms by Use Case:

Traversal → BFS / DFS

Cycle detection → DFS / Union-Find

Ordering → Topological Sort

Shortest Path → BFS (unweighted), Dijkstra (weighted)

MST → Kruskal / Prim

Connected Components → DFS / Union-Find

🧠 1️⃣0️⃣ Practice Problems

Problem	Pattern
BFS Traversal	BFS
DFS Traversal	DFS
Number of Islands	DFS
Course Schedule	Topological Sort
Detect Cycle (Undirected)	Union-Find
Shortest Path in Unweighted Graph	BFS
Network Delay Time	Dijkstra

Would you like me to include a **diagram (visual cheat chart)** showing:

- BFS vs DFS flow

- Topological sort order example
- Dijkstra priority queue expansion
  It makes remembering all graph patterns visually much easier for revision.

Perfect 🔥 You've now reached **Day 8 – Heaps & Greedy Algorithms**, a *goldmine* for interview questions (especially "Top K" and "Scheduling" patterns).

Here's your **cheat sheet + pattern recognition guide** that helps you quickly identify when to use **heap** or **greedy** approaches in interviews.

🧭 Day 8 – Heaps & Greedy Algorithms (Cheat Sheet)

🧱 1️⃣ Heap Basics

🔹 What is a Heap?

A **binary tree–based structure** (usually implemented as an array) where:

- **Min-Heap** → Parent ≤ Children
- **Max-Heap** → Parent ≥ Children

👉 It allows **O(log n)** insertion and deletion of min/max.

🔹 In Java:

```
// Min Heap
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max Heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

🧩 2️⃣ Common Heap Operations

Operation	Time Complexity
Insertion	O(log n)
Extract Min / Max	O(log n)
Peek	O(1)
Build Heap	O(n)

⚡ 3️⃣ When to Use Heap

Question Keyword	Think "Heap" if...
"Top K elements / frequent / largest / smallest"	Need to track *K-best or K-worst* items efficiently
"Running median / Kth smallest"	Need to maintain *dynamic order*
"Merge K sorted arrays / lists"	Need *sorted merging* efficiently
"Scheduling / deadlines"	Need *priority ordering*

💡 4️⃣ Top K Pattern (🔥 Most Common)

🔹 Example 1 — K Largest Elements

```
int[] findKLargest(int[] nums, int k) {
  PriorityQueue<Integer> pq = new PriorityQueue<>(); // min heap
  for (int n : nums) {
    pq.offer(n);
    if (pq.size() > k) pq.poll(); // keep only k largest
  }
  return pq.stream().mapToInt(i -> i).toArray();
}
```

🧠 When to use:

“Find Kth largest/smallest”, “Top K elements”, “Largest sum combinations”

🔹 **Example 2 — Top K Frequent Elements**

```
int[] topKFrequent(int[] nums, int k) {
   Map<Integer,Integer> freq = new HashMap<>();
   for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);

   PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[1]-b[1]);
   for (var e : freq.entrySet()) {
      pq.offer(new int[]{e.getKey(), e.getValue()});
      if (pq.size() > k) pq.poll();
   }

   int[] res = new int[k];
   for (int i = 0; i < k; i++) res[i] = pq.poll()[0];
   return res;
}
```

🧩 **Pattern Trigger Keywords:**

“Most frequent”, “Top K”, “Kth largest/smallest”, “Streaming input”
→ **Min/Max Heap**

🔹 **Example 3 — Kth Largest Element in Stream (Dynamic)**

Use heap of size K and adjust as elements come in.

LeetCode: *703. Kth Largest Element in a Stream*

🧮 5️⃣ **Kth Smallest / Largest Pattern Summary**

Problem Type	Data Structure	Comparator	Complexity
Kth **Largest**	Min-Heap (size K)	Ascending	O(n log K)
Kth **Smallest**	Max-Heap (size K)	Descending	O(n log K)

🧠 6️⃣ Greedy Algorithms — Core Idea

Make **locally optimal** choices at each step, hoping for a **global optimum**.

Use when:

- Choices are **independent**
- Problem has **optimal substructure**
- Sorting by a certain **criterion** helps (like start/end time, weight/value ratio, etc.)

⚙️ 7️⃣ Greedy Patterns Cheat Table

Problem Keyword	Greedy Idea	Sorting Criteria
“Intervals”, “Meetings”, “Non-overlapping”	Choose interval with earliest end time	By end
“Jobs with deadlines”, “Max profit scheduling”	Pick highest profit job that fits	By profit desc
“Minimum platforms”, “Merge intervals”	Sort start/end separately	By start
“Activity selection”	Always choose next with smallest finish	By finish
“Fractional knapsack”	Pick max value/weight ratio	By value/weight
“Huffman coding”	Pick smallest two frequencies repeatedly	Min-heap

---

🧩 8️⃣ Classic Interval Problem (Greedy)

🔹 Example — Non-overlapping Intervals

```
int eraseOverlapIntervals(int[][] intervals) {
   Arrays.sort(intervals, (a,b) -> a[1] - b[1]); // sort by end time
   int count = 0, end = Integer.MIN_VALUE;
   for (int[] iv : intervals) {
      if (iv[0] >= end) end = iv[1]; // non-overlapping
      else count++; // overlapping
   }
   return count;
}
```

🧠 When to use:

"Minimum intervals to remove", "Max non-overlapping meetings", "Activity scheduling" → **Greedy with sorting**

🔹 Example — Meeting Rooms II

Use **Min-Heap** to track meeting end times.

```
int minMeetingRooms(int[][] meetings){
   Arrays.sort(meetings, (a,b) -> a[0] - b[0]);
   PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap for end times
   for(int[] m : meetings){
      if(!pq.isEmpty() && pq.peek() <= m[0]) pq.poll();
      pq.offer(m[1]);
   }
   return pq.size();
}
```

🧠 **When to use:**

"Find minimum rooms / resources required" → **Heap of end times**

🧩 9️⃣ **Fractional Knapsack (Greedy)**

```
double fractionalKnapsack(int[] wt, int[] val, int W){
   Item[] items = new Item[wt.length];
   for(int i=0;i<wt.length;i++)
      items[i] = new Item(wt[i], val[i], (double)val[i]/wt[i]);

   Arrays.sort(items, (a,b)->Double.compare(b.ratio, a.ratio));
   double res = 0; int cap = W;
   for(Item it: items){
      if(cap >= it.wt) { cap -= it.wt; res += it.val; }
      else { res += cap * it.ratio; break; }
   }
   return res;
}
```

🧠 **Pattern Trigger Keywords:**

"Maximum profit with limited capacity" → **Greedy by ratio**

🧩 🔟 **Pattern Recognition Table**

Question Type	**Technique**
"Top K / Kth smallest/largest"	**Heap (PriorityQueue)**
"Merge K sorted lists"	**Min-Heap**
"Find most frequent / least frequent"	**Heap + HashMap**

Question Type	Technique
“Schedule jobs / non-overlapping intervals”	**Greedy with sorting**
“Assign resources / minimum rooms / platforms”	**Heap + Greedy**
“Maximize profit / ratio-based”	**Greedy by ratio (Knapsack)**

🧠 1️⃣ Practice Problems (must-solve)

Problem	Concept
Kth Largest Element	Min-Heap
Top K Frequent Elements	Min-Heap + HashMap
Meeting Rooms II	Min-Heap
Merge K Sorted Lists	Min-Heap
Task Scheduler	Max-Heap
Non-overlapping Intervals	Greedy
Activity Selection	Greedy
Fractional Knapsack	Greedy

Would you like me to make a **visual cheat chart** (Heap vs Greedy flow):

- “Top-K → Heap”
- “Scheduling → Greedy by end time”
- “Knapsack → Greedy by ratio”
  It’s great as a 1-page revision diagram for interviews.